

[Euron : DL세션 1주차 예습과제 - 딥러닝 파이토치 교과서 1, 2장 개념정리]

1. 머신 러닝과 딥러닝

1.1 인공지능, 머신 러닝과 딥러닝

인공지능 : 인간의 지능을 모방하여 사람이 하는 일을 컴퓨터가 할 수 있도록 하는 기술

- 인공지능 > 머신 러닝 > 딥러닝

머신 러닝 / 딥러닝

- 둘 다 데이터를 이용해 학습하고 결과를 예측/분류
- 차이점은 주로 **특성 추출을 사람이 하느냐, 컴퓨터가 하느냐**
- 머신 러닝 : 주어진 데이터를 사람이 먼저 처리(전처리), 데이터 특징을 사람이 추출해서 알고리즘에 넣음
- 일반적으로 점수/분류 등 숫자 형태의 결과
- 특정 문제에 맞게 만든 알고리즘은 다른 문제에 그대로 재사용하기 어려움
- 딥러닝 : 대량의 데이터를 신경망에 넣으면 중요한 특징을 컴퓨터가 스스로 찾아서 학습
- 같은 유형의 데이터를 분석할 때 학습된 모델 재사용 가능
- 머신 러닝보다 데이터 많이 필요, 학습 시간도 긴 편

▼ 표 1-1 머신 러닝과 딥러닝

구분	머신 러닝	딥러닝
동작 원리	입력 데이터에 알고리즘을 적용하여 예측을 수행한다.	정보를 전달하는 신경망을 사용하여 데이터 특징 및 관계를 해석한다.
재사용	입력 데이터를 분석하기 위해 다양한 알고리즘을 사용하며, 동일한 유형의 데이터 분석을 위한 재사용은 불가능하다.	구현된 알고리즘은 동일한 유형의 데이터를 분석하는 데 재사용된다.
데이터	일반적으로 수천 개의 데이터가 필요하다.	수백만 개 이상의 데이터가 필요하다.
훈련 시간	단시간	장시간
결과	일반적으로 점수 또는 분류 등 숫자 값	출력은 점수, 텍스트, 소리 등 어떤 것이든 가능

1.2 머신 러닝이란

머신 러닝 : 인공지능의 한 분야. 대용량 데이터에서 컴퓨터가 지식이나 패턴을 찾아 학습하고 예측하는 기술 : 크게 학습 단계 / 예측 단계로 나눌 수 있음

학습 단계 : 데이터 → 특성 추출 → 머신 러닝 알고리즘 → 모델 생성

예측 단계 : 새로운 데이터 → 특성 추출 → 학습된 모델 → 결과

특성 추출

- 데이터가 어떤 특징을 가지고 있는지 찾아냄
- 찾은 특징을 컴퓨터가 처리할 수 있도록 벡터 형태로 변환

데이터

- 머신 러닝 모델을 만들 때 사용하는 자료
- 실제 데이터의 특징을 잘 반영해야 함

- 특정 방향으로 편향되지 않은 데이터가 중요
- 훈련 데이터셋 : 모델 학습에 사용
- 검증 데이터셋 : 학습 중 모델 성능 확인
- 테스트 데이터셋 : 최종 모델 성능 평가

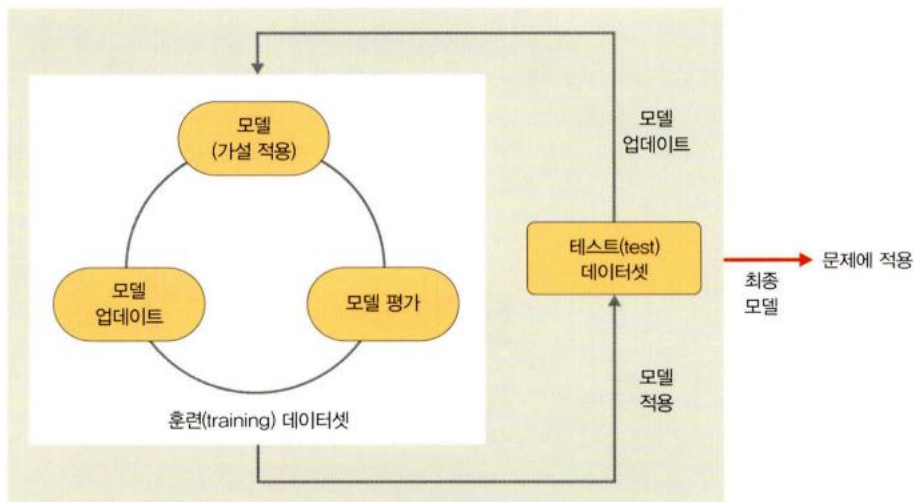
모델(가설)

- 머신 러닝 학습 단계에서 얻은 최종 결과물

모델 학습 절차

1. 모델 선택
2. 모델 학습 및 평가
3. 평가 결과를 바탕으로 모델 업데이트
4. 위 과정 반복

▼ 그림 1-5 머신 러닝의 문제 풀이 과정



1.2.2 머신 러닝 학습 알고리즘 : 지도 학습 / 비지도 학습 / 강화 학습

지도 학습

- 정답(label)을 알려 주고 학습시키는 방법

대표적으로

- 분류(classification) : 데이터를 정해진 범주로 구분
- 회귀(regression) : 연속적인 숫자값 예측

예) KNN, SVM, 결정 트리, 로지스틱 회귀, 선형 회귀 등

비지도 학습

- 정답을 알려 주지 않고 학습
- 데이터 자체의 특징을 보고 비슷한 것끼리 묶음
- 군집(clustering) : 특징 비슷한 데이터를 그룹으로 분류
- 차원 축소 : 데이터 정보는 최대한 유지하면서 차원 줄임

예) K-means, PCA 등

강화 학습

- 자신의 행동에 대한 보상을 받으면서 학습
- 보상을 많이 받을 수 있는 행동을 찾아감

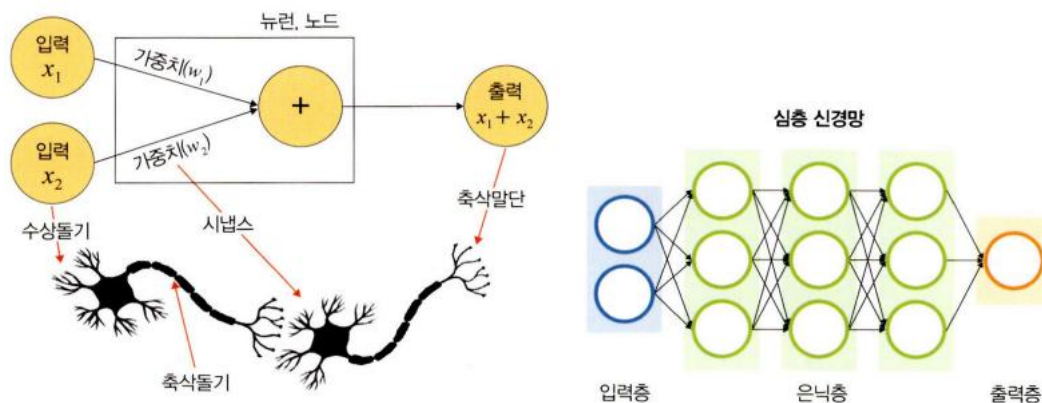
행동 → 보상 → 다음 행동 선택 → 반복

1.3 딥러닝이란

딥러닝 : 인간의 신경망 원리를 모방한 인공 신경망을 기반으로 만든 머신 러닝 방법의 일종

- 입력층(input layer)
- 은닉층(hidden layer)
- 출력층(output layer)
- 은닉층이 여러 개 있는 신경망 → 심층 신경망(DNN)
- 데이터셋에서 어떤 특성이 중요한지 스스로 학습 가능

▼ 그림 1-10 인간의 신경망 원리를 모방한 심층 신경망



1.3.1 딥러닝 학습 과정

1. 데이터 준비

- 파이토치/케라스에서 제공하는 데이터셋 사용
- 캐글처럼 공개된 데이터 사용
- 직접 수집한 데이터 사용 가능

2. 모델 정의

- 사용할 신경망 생성

3. 모델 컴파일

- 활성화 함수
- 손실 함수
- 옵티마이저 등 설정

4. 모델 훈련

- 훈련 데이터로 모델 학습
- 배치와 에포크 설정

배치(batch)

- 한 번에 모델에 넣어서 처리하는 데이터 묶음

배치 크기(batch size)

- 한 번에 처리하는 데이터의 개수

에포크(epoch)

- 전체 훈련 데이터를 한 번 모두 학습한 것
- epoch=10 → 전체 데이터를 10번 학습

5. 모델 예측

- 학습된 모델에 검증/테스트 데이터를 넣어 실제 결과 예측

역전파

- 예측값과 실제값 사이의 오차를 이용해서 가중치를 수정
- 출력 쪽에서 생긴 오차를 앞쪽으로 전달하면서 가중치 업데이트

1.3.2 딥러닝 학습 알고리즘

지도 학습

- 합성곱 신경망(CNN)

- 주로 이미지 데이터에 사용
- 이미지 분류 : 이미지가 어떤 클래스인지
- 이미지 인식 : 이미지 속 사물 종류 인식
- 이미지 분할 : 객체 사이 영역을 픽셀 단위로 구분
- 객체 탐지 등에도 사용

예) AlexNet, ResNet

- 순환 신경망(RNN)

- 시간 순서가 있는 데이터에 사용
- 시계열 / 자연어 / 음성 등에 사용

- LSTM

- RNN의 기울기 소멸 문제 등을 보완
- 망각 게이트 / 입력 게이트 / 출력 게이트 사용
- 필요한 정보를 오래 기억할 수 있도록 함

비지도 학습

워드 임베딩

- 단어나 문장을 숫자 벡터 형태로 표현
- 자연어 처리에 많이 사용

군집

- 정답 없이 특징이 비슷한 데이터끼리 묶음

전이 학습

- 이미 학습이 끝난 모델을 가져와서 새로운 문제에 사용

사전 학습 모델

- 우리가 풀 문제와 비슷한 데이터를 대량으로 이미 학습한 모델
- 그대로 사용하거나
- 원하는 문제에 맞게 일부 다시 학습 → 미세 조정(fine-tuning)

예) BERT, MobileNetV2 등

2. 실습 환경 설정과 파이토치 기초

2.1 파이토치 개요

파이토치(PyTorch)

- 딥러닝 모델 개발을 위한 프레임워크
- GPU에서 텐서 연산 가능
- 동적 신경망 구축 가능
- 비교적 직관적으로 사용 가능

GPU

- 연산을 빠르게 처리하기 위해 사용
- 한꺼번에 많은 계산을 하는 데 유리해서 딥러닝 학습에 많이 사용

텐서

- 파이토치에서 사용하는 기본 데이터 형태
- NumPy의 ndarray와 비슷
- GPU 연산 가능

동적 신경망

- 파이토치는 Define by Run 방식
- 실행하면서 계산 그래프가 만들어짐
- 학습을 반복할 때 네트워크 구조 변경 가능

2.1.1 파이토치 특징 및 장점

벡터 / 행렬 / 텐서

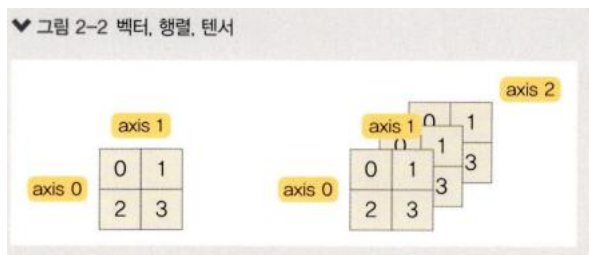
- 인공지능에서 데이터는 주로 벡터 형태로 표현
- 벡터 : 1차원
- 행렬 : 2차원
- 텐서 : 행렬을 확장한 다차원 형태
- axis 0 : 첫 번째 축
- axis 1 : 두 번째 축
- axis 2 : 세 번째 축

행렬

- 여러 차원을 가지는 데이터 레코드의 집합

텐서

- 행렬의 다차원 표현



2.1.2 파이토치의 아키텍처

주요 패키지

- torch : GPU를 지원하는 텐서 패키지
- torch.autograd : 자동 미분 패키지
- torch.nn : 신경망 구축 및 훈련 패키지
- torch.multiprocessing : 파이썬 멀티프로세싱 패키지
- torch.utils : DataLoader 및 기타 유틸리티 제공

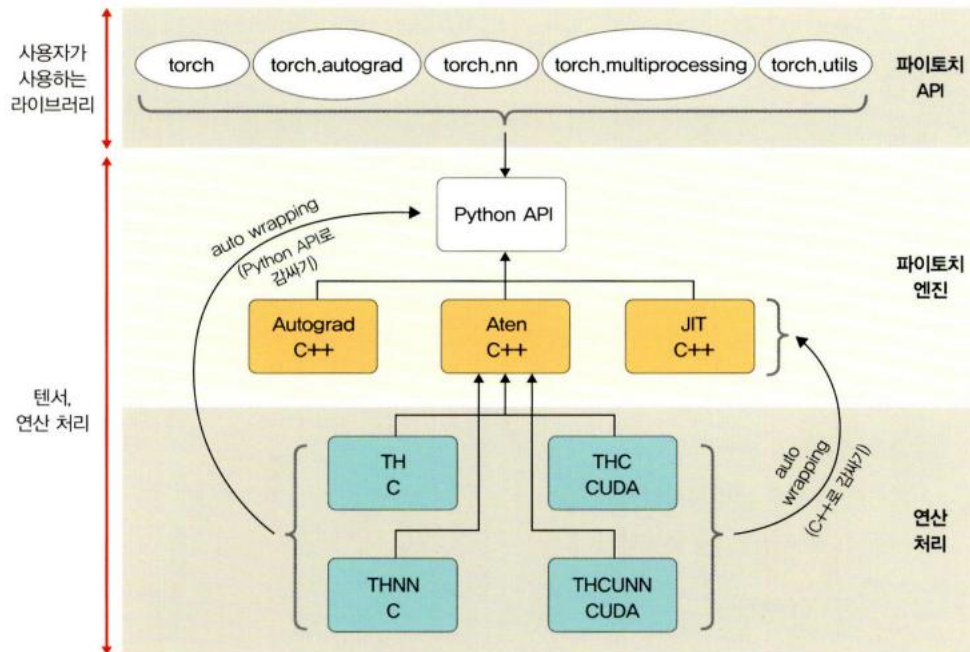
autograd

- 신경망 학습에 필요한 기울기를 자동으로 계산

nn

- 신경망 계층, 활성화 함수 등 제공
- 모델 만들 때 많이 사용

▼ 그림 2-4 파이토치의 아키텍처



2.2 파이토치 기초 문법

2.2.1 텐서 다루기

텐서 생성 및 변환

- `torch.tensor()`를 이용해 텐서 생성
- `device`를 지정해 CPU/GPU에서 사용 가능
- `dtype`을 이용해 데이터 자료형 지정 가능
- Tensor → NumPy 변환 가능
- GPU에 있는 Tensor를 NumPy로 바꾸려면 먼저 CPU로 이동

텐서의 인덱스 조작

- 파이썬 리스트처럼 인덱싱/슬라이싱 가능

예)

- `[0]` : 첫 번째 값
- `[-1]` : 마지막 값
- `[2:5]` : 특정 범위

텐서 자료형

- `torch.FloatTensor` : 32비트 부동소수점
- `torch.DoubleTensor` : 64비트 부동소수점
- `torch.LongTensor` : 64비트 부호 있는 정수

텐서 연산

- 덧셈, 뺄셈 등 기본적인 연산 가능

- 같은 위치의 값끼리 연산

텐서 차원 조작

view() : 텐서가 가진 값은 그대로 두고 모양(shape)을 변경

- -1 사용 → 나머지 크기를 자동으로 계산

2.2.2 데이터 준비

파이토치에서 데이터를 준비하는 방법

- 파일을 직접 불러와 사용
- 커스텀 데이터셋 생성
- 파이토치에서 제공하는 데이터셋 사용

CSV 파일 사용

- pandas의 read_csv()로 데이터 불러옴
- 불러온 데이터를 텐서 형태로 바꿔서 모델에 사용

CustomDataset

직접 데이터셋을 만들 때 Dataset을 상속해서 사용

__init__()

- 필요한 변수 선언
- 데이터 불러오기/전처리

__len__()

- 전체 데이터셋 길이(샘플 수) 반환

__getitem__()

- index에 해당하는 데이터 하나를 가져옴

DataLoader

- Dataset에서 데이터를 배치 단위로 가져오는 역할

batch_size

- 한 번에 가져오는 데이터 개수

shuffle=True

- 데이터 순서를 섞어서 사용

파이토치 제공 데이터셋

- torchvision에서 MNIST 등 여러 데이터셋 제공
- transforms를 이용해 전처리 가능

예)

- ToTensor() : 텐서로 변환
- Normalize() : 데이터 정규화

2.2.3 모델 정의

파이토치에서 모델 정의 방법

- 단순 신경망
- nn.Module 상속
- nn.Sequential
- 함수 이용

nn.Module

직접 신경망 모델을 만들 때 기본적으로 상속

`__init__()`

- 사용할 계층 정의

`forward()`

- 입력값이 계층들을 어떤 순서로 지나가는지 정의

Linear

- 선형 계층
- 입력값에 가중치와 편향을 적용

Sequential

- 여러 계층을 순서대로 묶어서 사용

예) Conv2d → ReLU → MaxPool2d → Linear

children() / modules()

- `model.children()` : 바로 아래에 있는 하위 모듈 확인
- `model.modules()` : 모델 안의 전체 모듈 확인

2.2.4 모델의 파라미터 정의

모델을 학습시키기 위해 필요한 것

- 손실 함수
- 옵티마이저
- 학습률
- 학습률 스케줄러

손실 함수(loss function)

- 실제값과 모델의 예측값 차이를 계산
- 모델이 얼마나 틀렸는지 수치로 나타냄

예) BCELoss, CrossEntropyLoss, MSELoss

옵티마이저(optimizer)

- 손실이 줄어드는 방향으로 모델의 가중치를 업데이트

예) SGD, Adam, Adagrad, RMSProp 등

학습률(learning rate)

- 한 번 가중치를 수정할 때 얼마나 크게 수정할지 결정
- 너무 크거나 너무 작으면 학습 잘 안 될 수 있음

학습률 스케줄러

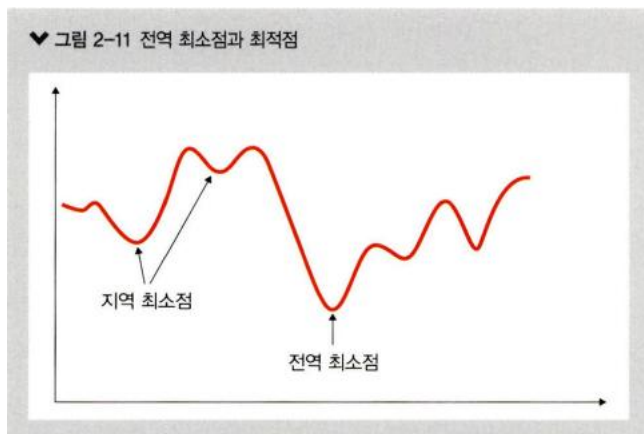
- 학습이 진행되는 동안 학습률을 조절

예) LambdaLR, StepLR, MultiStepLR, ExponentialLR, CosineAnnealingLR, ReduceLROnPlateau

지역 최소값 / 전역 최소값

- 지역 최소값 : 특정 부분에서 가장 작은 값
- 전역 최소값 : 전체에서 가장 작은 값

→ 손실 함수의 전역 최소값을 찾는 것이 목표



2.2.5 모델 훈련

파이토치 학습 과정

1. 모델에 데이터 입력
2. 예측값 계산
3. 손실 계산
4. 기존 기울기 초기화
5. 역전파
6. 가중치 업데이트

optimizer.zero_grad()

- 이전에 계산된 기울기 초기화

loss.backward()

- 역전파 수행
- 각 파라미터의 기울기 계산

optimizer.step()

- 계산된 기울기를 이용해 가중치 업데이트

2.2.6 모델 평가

- 훈련한 모델을 테스트 데이터로 평가
- 교재에서는 torchmetrics 이용

정확도(accuracy)

- 전체 데이터 중 맞게 예측한 데이터 비율

2.2.7 훈련 과정 모니터링

TensorBoard

- 모델이 학습되는 과정을 그래프로 확인
- 손실값 등의 변화 확인 가능

model.train()

- 모델을 훈련 모드로 설정

model.eval()

- 모델을 평가 모드로 설정

torch.no_grad()

- 기울기 계산 X
- 모델 평가할 때 불필요한 계산 줄임

2.3 실습 환경 설정

2.3.1 아나콘다 설치

Anaconda

- 파이썬과 여러 데이터 분석 패키지를 편리하게 설치/관리할 수 있도록 해주는 배포판

2.3.2 가상 환경 생성 및 파이토치 설치

가상 환경

- 프로젝트별로 Python/라이브러리 버전을 따로 관리하는 환경
- 프로젝트마다 필요한 패키지가 다를 수 있기 때문에 사용
- 라이브러리 버전 충돌 방지

2.4 파이토치 코드 맛보기

- 자동차 데이터를 이용해서 간단한 분류 모델 생성
- 사용 데이터 : car_evaluation.csv

데이터 변수

- price : 자동차 가격
- maint : 자동차 유지 비용
- doors : 자동차 문 개수
- persons : 탑승 인원
- lug_capacity : 수하물 용량
- safety : 안전성
- output : 자동차 평가 결과

사용 라이브러리

- torch
- torch.nn
- numpy
- pandas
- matplotlib
- seaborn
- scikit-learn

데이터 불러오기

- pd.read_csv()로 csv 파일 불러옴
- head()로 데이터 일부 확인

데이터 시각화

- output 값을 이용해 자동차 평가 결과의 분포 확인
- pie chart 사용

범주형 데이터 변환

- 자동차 데이터 대부분이 범주형 데이터

예)

price / maint / doors / persons / lug_capacity / safety

- 신경망에 바로 넣을 수 없으므로 숫자 형태로 변환

astype('category')

→ 해당 열을 범주형 데이터로 변환

cat.codes

→ 각 범주를 숫자로 변환

np.stack

- 여러 배열을 하나로 합침
- 여러 범주형 변수들을 하나의 배열로 만들

이후 torch.tensor()를 이용해서 Tensor로 변환

get_dummies()

- 범주형 데이터를 더미 변수 형태로 변환
- 각 범주를 0/1 형태로 표현

임베딩(Embedding)

- 범주형 데이터를 일정한 크기의 벡터로 변환
- nn.Embedding 사용
- 범주의 개수에 따라 임베딩 크기를 정함
- 원래 범주형 데이터를 그대로 사용하는 것보다 신경망에서 처리하기 편하게 만들

훈련 데이터 / 테스트 데이터 분리

- 전체 데이터 중 약 20%를 테스트 데이터로 사용
- 나머지는 훈련 데이터

모델 생성

모델에서 사용하는 주요 계층

Embedding

- 범주형 데이터를 벡터로 변환

Linear

- 선형 계층

ReLU

- 활성화 함수
- 비선형성 추가

BatchNorm1d

- 배치 정규화

Dropout

- 일부 뉴런을 랜덤하게 제외
- 과적합 방지에 사용

손실 함수 / 옵티마이저

CrossEntropyLoss

- 여러 클래스 중 하나를 예측하는 분류 문제에서 사용

Adam

- 가중치를 업데이트하는 옵티마이저

CPU / GPU 사용

- `torch.cuda.is_available()`로 GPU 사용 가능한지 확인

GPU 가능 → CUDA 사용

GPU 불가능 → CPU 사용

모델 학습

에포크마다 예측 → 손실 계산 → 기울기 초기화 → 역전파 → 가중치 업데이트 반복

- 일정 에포크마다 loss 출력해서 학습 상태 확인

모델 테스트

- 학습 완료 후 테스트 데이터 넣어서 모델 평가
- 평가할 때 `torch.no_grad()` 사용

분류 모델 평가

`sklearn.metrics` 사용

- confusion matrix
- accuracy
- precision
- recall
- F1-score

혼동 행렬(Confusion Matrix)

실제값과 모델 예측값 비교

- True Positive(TP) : 실제 1 / 예측도 1
- True Negative(TN) : 실제 0 / 예측도 0
- False Positive(FP) : 실제 0 / 예측은 1 → Type I error
- False Negative(FN) : 실제 1 / 예측은 0 → Type II error

정확도(Accuracy)

전체 데이터 중 맞게 예측한 비율

$$(TP + TN) / (TP + TN + FP + FN)$$

재현율(Recall)

실제 Positive 중에서 Positive라고 제대로 예측한 비율

$$TP / (TP + FN)$$

정밀도(Precision)

Positive라고 예측한 것 중 실제 Positive인 비율

$$TP / (TP + FP)$$

F1-score

- 정밀도와 재현율을 같이 고려한 지표
- 둘의 조화평균

$$2 \times (Precision \times Recall) / (Precision + Recall)$$